MSOT STUDENT CHALLENGE - PROBLEM 1

Mirai Course Progression Tracker

Architecture, design, verification and demonstration report

239	automated tests
2,100	load requests
0%	load errors

Prepared 8 October 2026

Status: source and documentation verified; public deployment and external field study pending

This report uses only fictional demonstration identities and non-sensitive sample data.

Executive overview

Mirai Tracker replaces spreadsheet-based syllabus monitoring with a calendar-aware, role-scoped system. It combines semester setup, timetable capacity, lesson plans, daily faculty logs, weighted progress, exam readiness, alerts and exports in one auditable workflow.

4	user roles
30	domain models
13/13	security controls
11/11	deployment checks

What the system decides

- What should have been covered by a selected date.
- What was actually covered, weighted by planned topic hours.
- Whether future timetable capacity can finish the remaining syllabus.
- Which subjects, campuses and exams need attention first.

Verified baseline

All workspace type checks, 239 automated tests and production application builds pass. Three hand calculations match to two decimals. The local four-campus, 30-subject load baseline completed 2,100 requests with zero errors at up to 200 concurrent users.

Honest completion boundary

The production package is configuration-ready, but this report does not claim a live URL. Docker-backed database tests, backup restoration, the public deployment check and the five-faculty phone study require infrastructure or people supplied by the deployer.

Problem, goals and users

The operating problem is not simply recording completed topics. Academic managers need a reliable early-warning view that accounts for working days, holidays, cancellations, topic size, exam scope and the teaching hours still available.

Role model

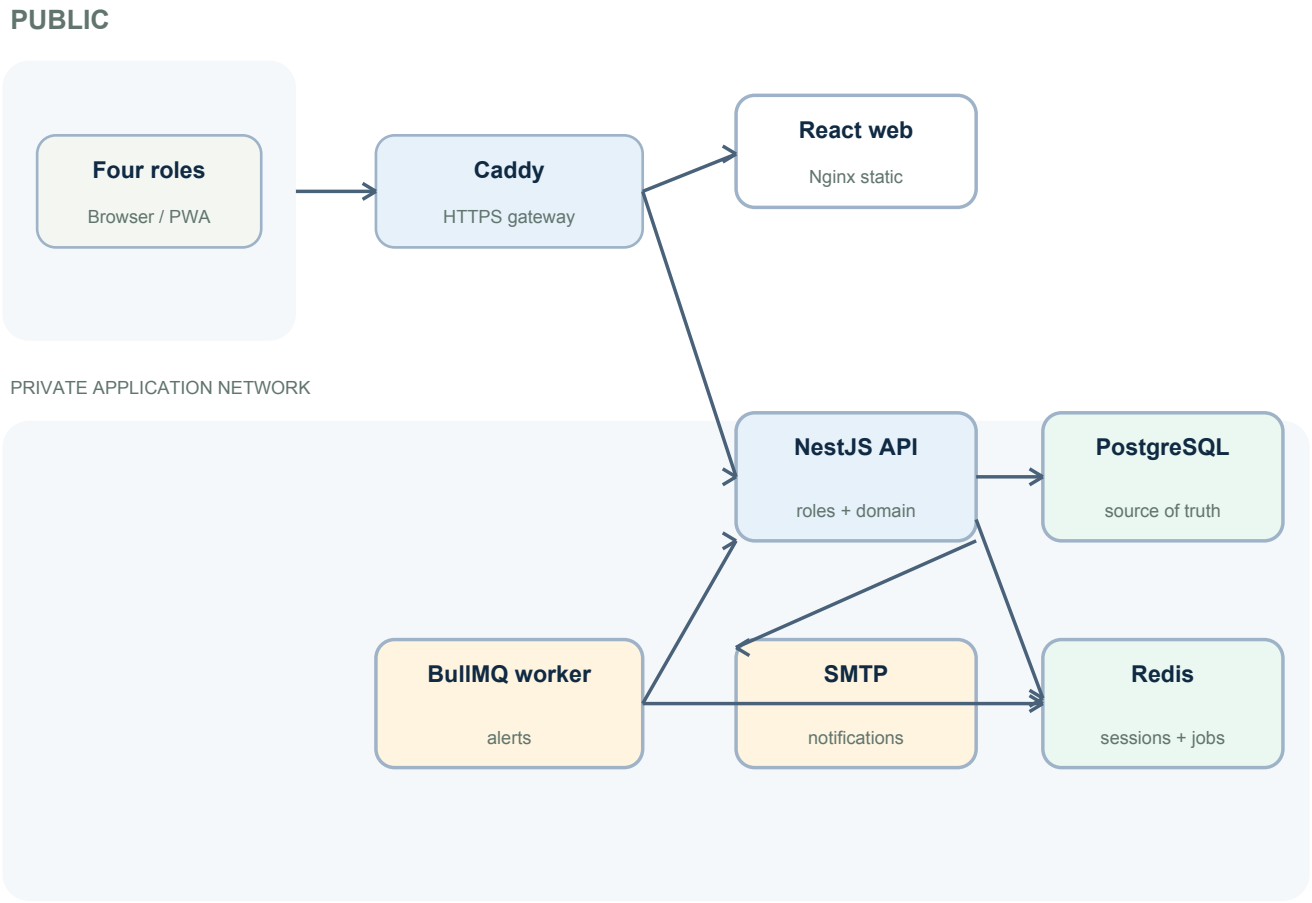
Role	Capabilities	Enforced scope
Program Manager	Academic setup, imports, all dashboards, audit and exports	All campuses
Campus Manager	Timetable, campus dashboards and escalations	Assigned campus
Faculty	Lesson plans, assigned daily logs and subject view	Active assignments
Student	Covered, next and exam-scope information	Own batch; read-only

Design principles

- Minutes are canonical; hours are display values.
- Permissions come from server-side identity and assignment records.
- History is versioned or revisioned instead of silently overwritten.
- Every risk status remains explainable from syllabus and timetable inputs.

Architecture

A React PWA serves desktop managers and phone-first faculty. The NestJS API owns validation, access policy and domain transactions. PostgreSQL is the durable source of truth; Redis holds sessions and BullMQ state.



Caddy is the only public container. It terminates HTTPS and routes /api to NestJS while Nginx serves the PWA. PostgreSQL and Redis have private persistent volumes.

Core workflows

Stage	Input	System result
Academic setup	Campus, semester, Saturdays, dates and exams	One explainable working calendar
Bulk import	Validated CSV/XLSX rows	Atomic campus-batch-subject graph
Timetable	Weekly slots and assignments	Dated sessions and planned minutes
Lesson plan	Units, topics, minutes and exam mappings	Versioned syllabus scope
Planned dates	Topics plus real sessions	Versioned start/finish dates and shortfall
Daily logging	Faculty confirmation or correction	Revision, progress and audit record
Monitoring	Selected date	Subject, track, campus and exam risk
Operations	Missing logs and export requests	Idempotent alerts and audited reports

Change handling

A holiday is removed before session generation. A later cancellation removes that session's capacity and creates a new planned-date version. A corrected faculty log creates a new revision and recalculates topic progress atomically. These choices keep historical evidence while allowing the current view to update.

Progress calculation

Every formula operates on integer minutes and converts to hours only for display.

Metric	Definition
Hours in scope	Sum of planned minutes for all non-skipped topics / 60
Covered	Completed minutes + in-progress minutes x completion percentage
Coverage	Covered / in-scope
Remaining	In-scope - covered
Available	Future timetable minutes through target finish date
Buffer	Available / remaining
Pace	Covered - planned coverage expected by the selected date

Hand-calculated evidence

Subject	Coverage	Remaining	Available	Buffer	Pace	Status
CS-DSA	58.33%	2.50 h	3.00 h	1.20	-0.50 h	ON TRACK
MATH-101	41.67%	3.50 h	3.75 h	1.07	-0.50 h	TIGHT
PHY-201	38.89%	5.50 h	4.00 h	0.73	-1.00 h	DANGER

Default thresholds: ON TRACK >= 1.15, TIGHT >= 1.00, DANGER < 1.00. Campus and track status preserves the worst subject status.

Data and audit design

Thirty PostgreSQL models cover five connected areas. Foreign keys, compound uniqueness, enum states and SQL constraints complement request validation.

Area	Representative records	History strategy
Identity	Users, credentials, roles, staff and students	Role assignments remain explicit
Academic	Campuses, tracks, semesters, batches and offerings	Imports are atomic and recorded
Planning	Calendars, exams, assignments, timetable and sessions	Generated sessions preserve logged history
Curriculum	Plans, units, topics, exam scope and schedules	Plans and schedules are versioned
Delivery	Session logs and topic updates	Corrections create revisions
Operations	Snapshots, alerts, delivery, exports and audit	Events and outcomes are retained

Auditability

Successful logins and domain mutations create AuditEvent records. Admin and Program Manager roles can query bounded history by action, entity, actor, campus and time. The application exposes no audit update or delete route.

Security and recovery

Control	Implementation
Credentials	scrypt password hashes with individual salts
Sessions	256-bit opaque token; HMAC-derived Redis key; HTTP-only cookie
Authorization	Global authentication and role guards plus resource-level scope checks
Browser writes	Exact Origin match; SameSite cookie; exact credentialed CORS
Abuse	15-minute failed-login counter with generic account response
Validation	Zod request schemas, upload limits and PostgreSQL constraints
Errors	Expected errors preserved; unexpected details hidden from clients
Backups	Custom-format dump, restricted permissions and SHA-256 checksum
Recovery	Disposable restore check and confirmation-gated real restore

13/13	M17 controls pass
0	known secret patterns
15 min	login window

Test strategy and results

170	API tests
43	web tests
26	worker + shared
239	total

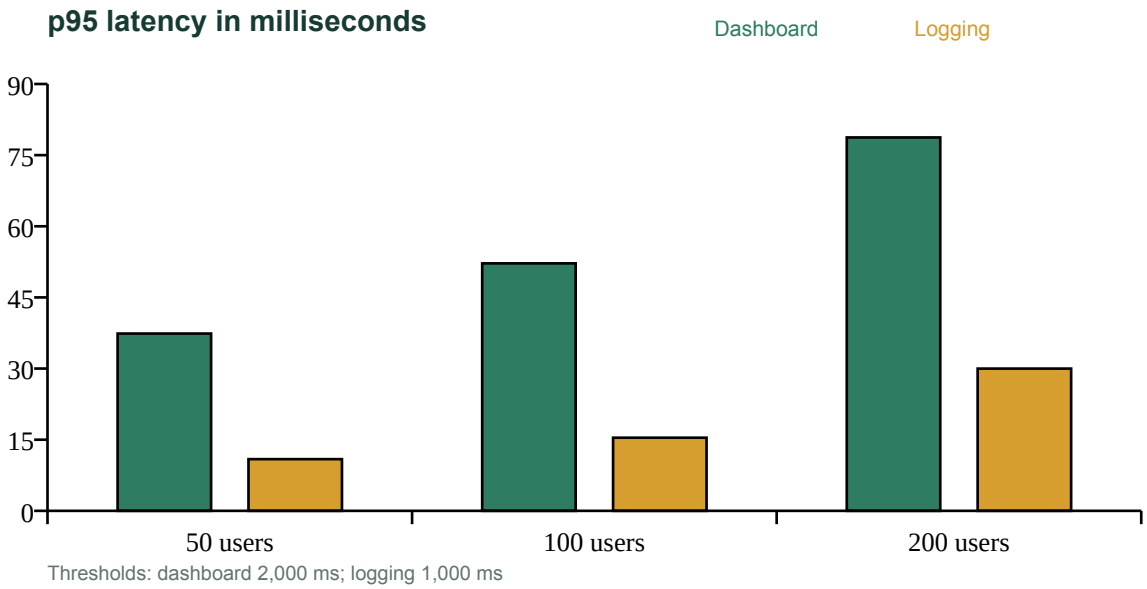
Test layer	What it proves	Current result
Pure functions	Calendar, timetable, topic allocation and progress formulas	Pass
Services	Permissions, transactions, reminders, imports and exports	Pass
React	Role-aware screens, forms and read-only behavior	Pass
Schema contracts	Models, relations and expected SQL safeguards	Pass
Database integration	Real PostgreSQL constraints and migrations	Pending runtime
Field usability	Five faculty on phones under 60 seconds	Pending volunteers
Public deployment	HTTPS, live dependencies, SMTP and cloud latency	Pending M18 URL

Evidence discipline

Unavailable evidence remains labelled pending. The report does not convert prepared scripts, screenshots or local mocks into claims about a live database, real volunteers or a public deployment.

Performance baseline

The M16 harness ran dashboard reads and end-of-day logging at 50, 100 and 200 concurrent users with three measured iterations per virtual user. The profile represented four campuses and 30 subjects.



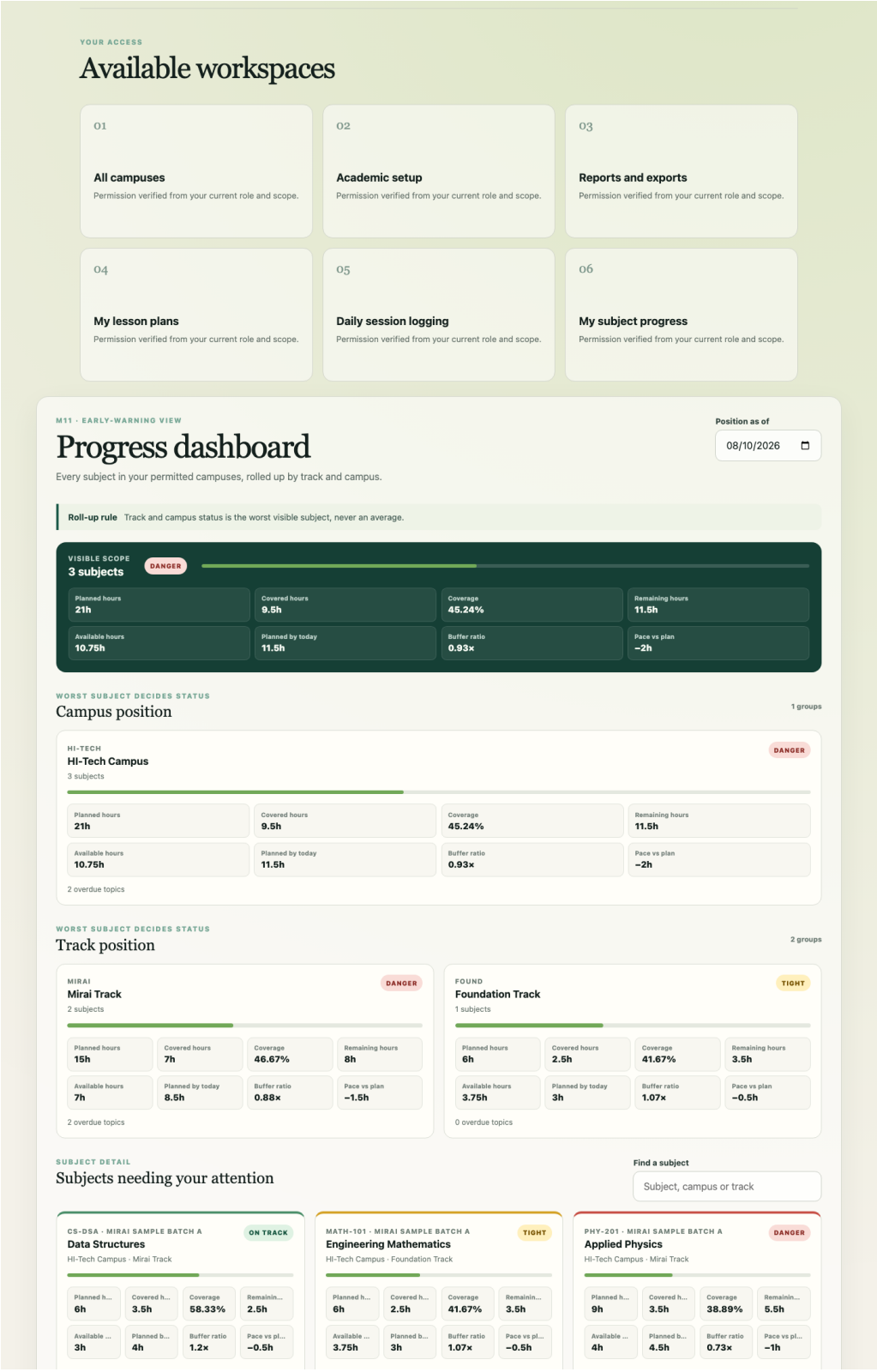
Thresholds: dashboard 2,000 ms; logging 1,000 ms

2,100	measured requests
0	errors
78.73 ms	max dashboard p95
29.99 ms	max logging p95

This local deterministic baseline excludes PostgreSQL, Redis, TLS and network latency. It proves the acceptance harness and calculation path, while the same authenticated workload must be rerun on the M18 deployment.

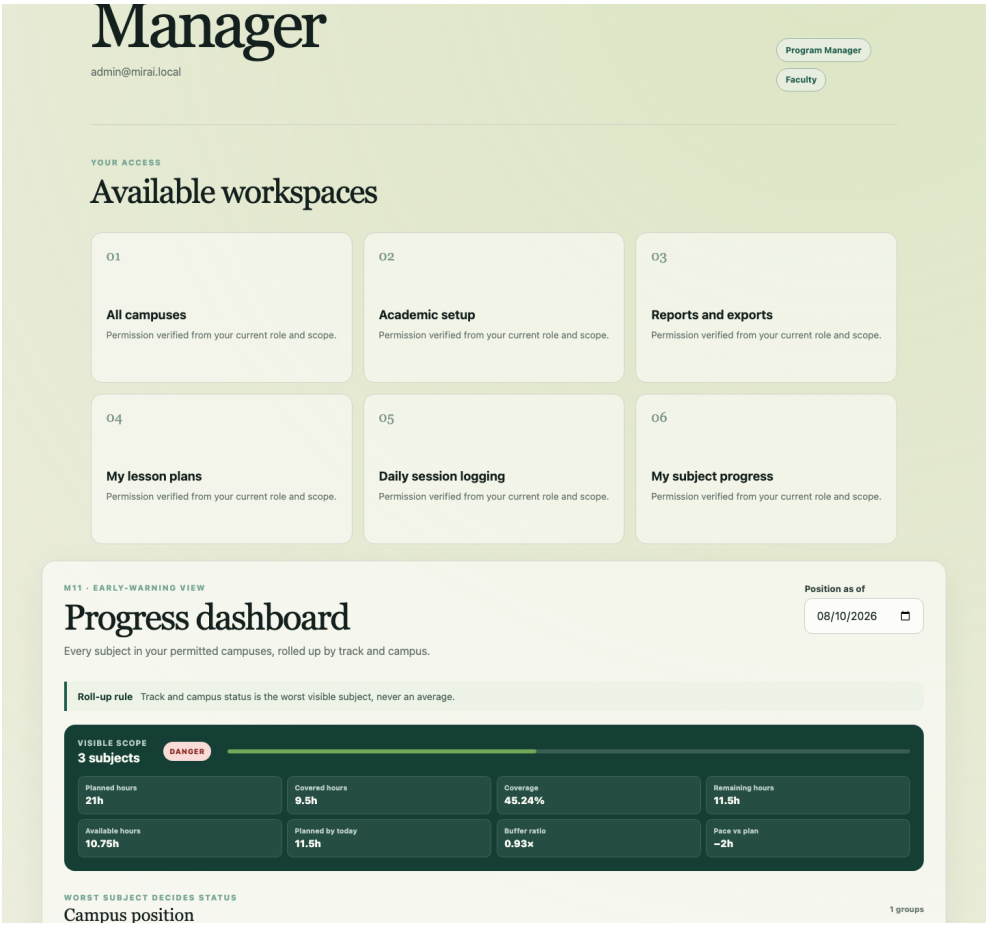
Interface evidence - manager

The saved manager workspace demonstrates the dashboard and the connected semester, import and timetable workspaces.

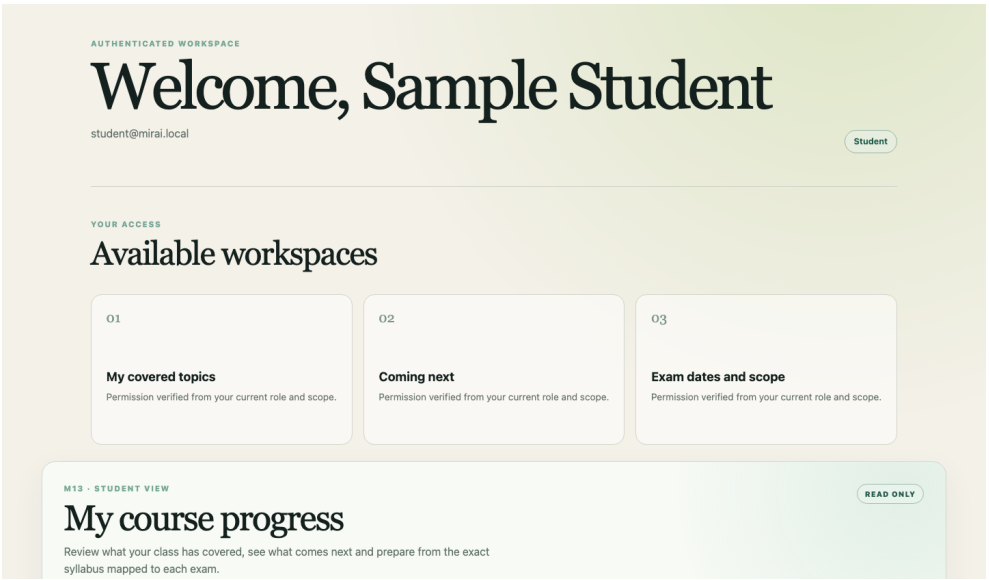


M11 manager dashboard: subject, track and campus risk

Interface evidence - exam and student



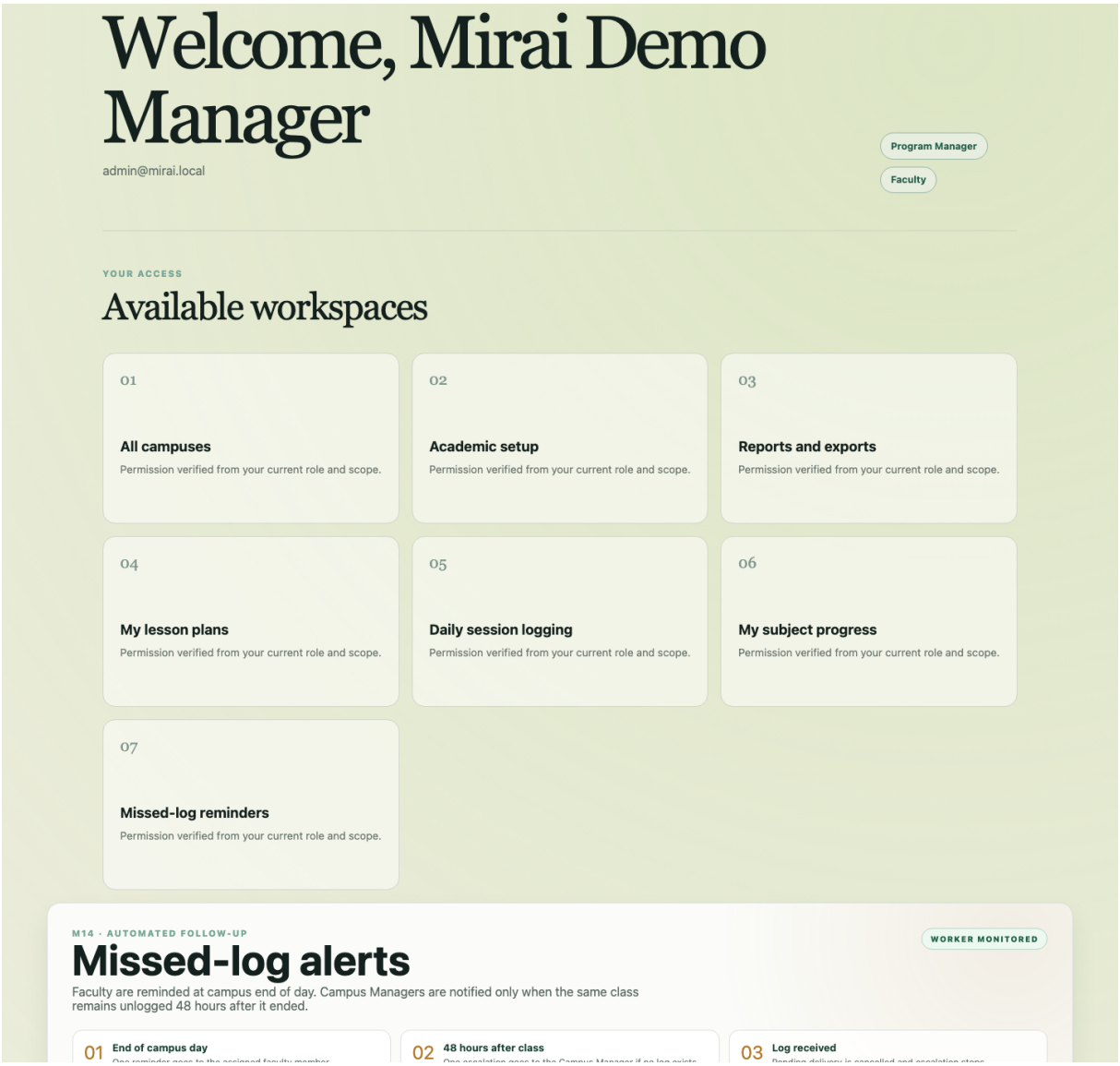
M12 exam readiness with independent scope



M13 student batch-only view

Interface evidence - alerts

The alert center makes faculty reminders and Campus Manager escalations visible alongside SMTP delivery state.

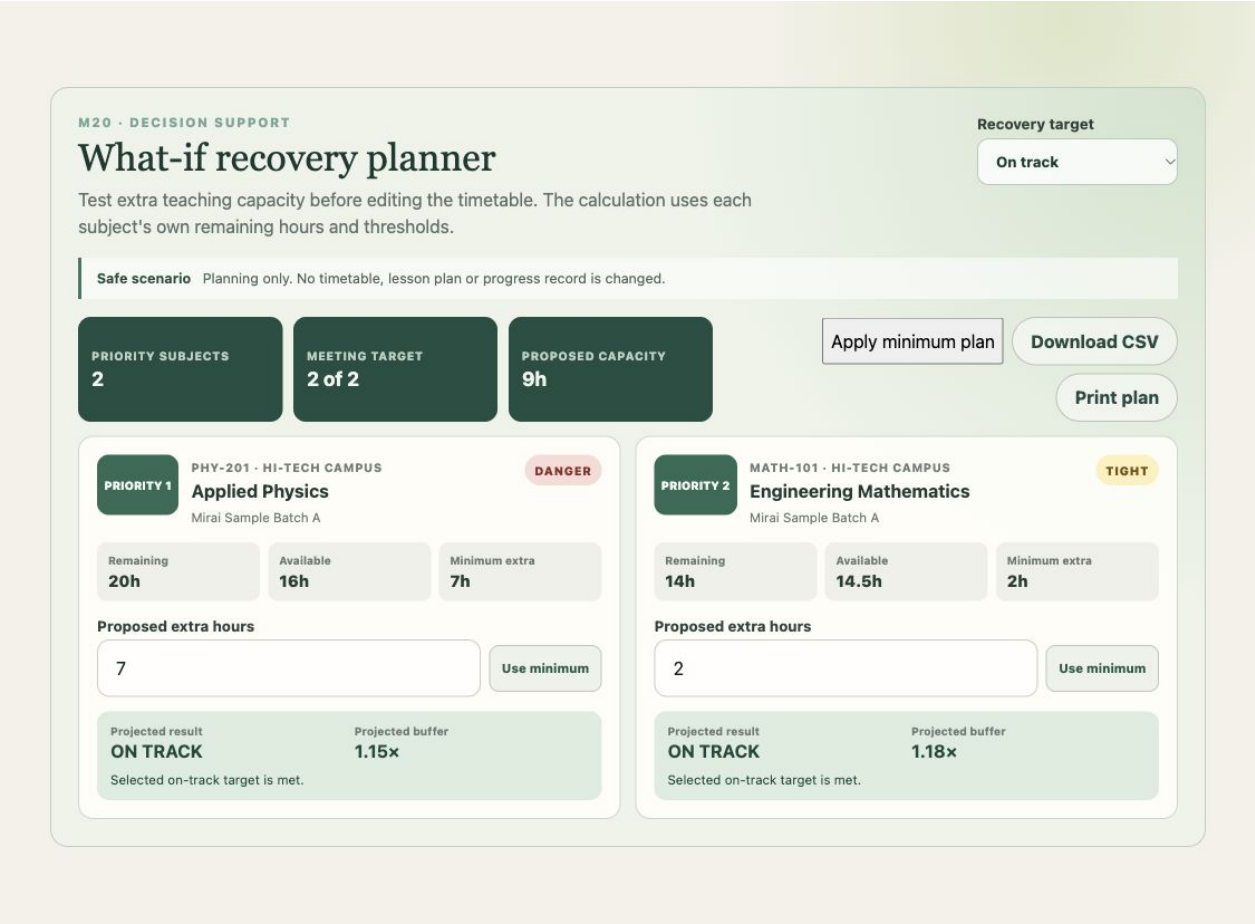


M14 missed-log reminders and 48-hour escalation

Stretch feature - recovery planner

M20 turns an early warning into a concrete intervention. Managers and faculty can select a TIGHT or ON TRACK goal, see the highest-risk subjects first and test additional teaching capacity without changing live academic records.

8/8 controls	7 focused tests	0 database writes	0.5 h increment



Applied minimum plan: two priority subjects reach ON TRACK

Explainable and safe

Required capacity equals remaining hours multiplied by the selected subject threshold. The exact shortfall is rounded up to a schedulable half hour. The planner uses the role-scoped M11 response, performs no database writes and keeps the official M10 thresholds visible through projected ratio and status.

Production deployment

The M18 production package builds three targets: Nginx web, NestJS API and BullMQ worker. Caddy publishes HTTPS. PostgreSQL and Redis remain private and persistent. Health checks order startup and the API applies migrations before serving.

Step	Command or outcome
Configure	Copy .env.production.example and supply independent secrets
Validate	pnpm deploy:config and pnpm verify:m18
Start	pnpm deploy:up
Seed	One-time DEMO_USER_PASSWORD process; not stored in production env
Verify data	pnpm deploy:verify-data
Verify public	pnpm verify:m18:live with transient credentials

11/11	deployment controls
HTTPS	automatic gateway
PRIVATE	data services

Scaling path

Move PostgreSQL and Redis to managed services, add point-in-time recovery, use object storage for uploaded files and run multiple stateless API replicas. Keep one scheduler owner until distributed job-scheduler coordination is defined.

Demonstration plan

Minute	Persona	Proof point
0-2	Program Manager	Calendar, exams and validated semester import
2-3	Campus Manager	Weekly slots generate dated teaching sessions
3-6	Faculty	Lesson plan, planned dates and pre-filled daily log
6-8	Manager	Weighted dashboard, recovery scenario and exam readiness
8-9	Student	Own batch, covered and next topics, no editing
9-10	Manager	Alerts, exports, evidence and close

Core explanation

Mirai Tracker does not count topics. It weights planned minutes and compares the remaining syllabus with the real teaching capacity left after holidays and cancellations.

Fallback

If the live environment is unavailable, show the saved M11-M14 screenshots and run the evidence verifiers. State clearly that this is the fallback and do not describe it as a live deployment.

Submission readiness

Complete

- Source, migrations, seed, local and production configuration.
- Architecture, design, Test Report, demo script and evidence index.
- 239 automated tests, type checks and production application builds.
- Calculation, load, security, deployment and stretch-feature verifiers.

Close before a fully live claim

- Choose the server/account, point DNS, configure SMTP and record the M18 URL.
- Run migrations, the demo verifier, PostgreSQL integration tests and restore drill.
- Repeat M16 against the authenticated deployed stack.
- Collect five real faculty phone timings and calculate the median.
- Confirm college deadline, file naming and demonstration date.

Conclusion

The implementation provides a coherent, explainable foundation for multi-campus course progression. Its strongest property is traceability: timetable capacity, syllabus scope, delivery logs, calculations, alerts and evidence all connect through durable records. Final external verification remains visible rather than being overstated.

Evidence index

Evidence	Purpose
m10-hand-calculations.csv	Three two-decimal subject calculations
m11-progress-dashboard.png	Subject, track and campus dashboard
m12-exam-readiness.png	Independent exam scope and hours needed
m13-student-read-only-view.png	Batch-only student experience
m14-missed-log-alerts.png	Reminder and escalation interface
m16-load-test-results.json/csv	Concurrency, p50, p95, p99 and errors
m17-security-checklist.json/csv	Security and recovery control record
m18-deployment-checklist.json/csv	Production-package control record
m19-submission-checklist.json/csv	Documentation completeness record
m20-stretch-checklist.json/csv	Recovery-planner control record

Reproduction

Run: pnpm check; pnpm verify:m10; pnpm verify:m16; pnpm verify:m17; pnpm verify:m18; pnpm verify:m19; pnpm verify:m20.

End of report